3 Program Logic: Loops, Selection, and Exception handling

For Loop

Create a program called **for_loop** that prints integers between 0 and 9 on single line as shown in the example output. **NB!** You should use the *for loop* in this exercise.

Example output:

```
0 1 2 3 4 5 6 7 8 9
```

Print Numbers

Create a program called **print_numbers** that prints integers from 10 to 30 to the console. The program should print the integers on a single line. You should use a ***for loop*** in this exercise.

Example output:

```
10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30
```

Print Descending

Create a program called **print_descending** that first inputs an integer from the user. Then, the program should print all integers between one and the inputted integer in descending order. The integers should be printed on a single line. You should use the ***for loop*** in this exercise.

If there are no integers to be printed, then the program should print "**Nothing to be printed**". Otherwise, the program should also print the sum of the integers as below.

Example output:

```
Enter an integer: 12
12 11 10 9 8 7 6 5 4 3 2 1
The sum is 78
```

Compute Average

Create a program called **compute_average** that first inputs decimal values from the user until he/she decides to quit by entering zero. Then the program should compute and display the average of the entered values (excluding the zero) with two decimals. If the only entered value is zero, then the program should print "**Nothing to be calculated**".

Example output:

```
Enter first number: 5.2
Enter next number: 3.1
Enter next number: 0
The average is 4.15
```

Please notice the ***standard while loop pattern*** below and use it consistently to write clean code.

```
get first value
while (...) {
    process the value
    get next value
}
```

NB! Please use the standard while loop pattern in this exercise. That is,

1. Do not use the break statement.
2. Do not decrement a counter.

Check Integer

Create a program called **check_integer** that asks the user to enter an integer. If the inputted value cannot be converted to an integer, the program should print the inputted text enclosed in single quotes and "**is not an integer**". Otherwise, the program should print "**OK**".

NB! Please use exception handling in this exercise.

Example output:

```
Enter an integer: ten
'ten' is not an integer
```

Hint: To be able to print an invalid value, you should *assign the inputted string to a separate variable* before trying to convert it to an integer.

Check Weekday Number

Create a program called **check_weekday_number** that inputs a weekday number from the user. If the inputted value is not valid weekday number, the program should print "**Please enter an integer between 1 and 7**". Otherwise, the program should print "**OK**".

NB! Please use exception handling in this exercise.

Example output:

```
Enter a weekday number: -1
Please enter an integer between 1 and 7
```

Check Month

Create a program called **check_month** that inputs a month number from the user. If the input is a valid month number, the program should print "**OK**". Otherwise, the program should print the entered value and "**is not a valid month number**" and prompt a month number again. The program should keep asking month numbers until the user enters a valid month number.

NB! Please use exception handling in this exercise.

Example output:

```
Enter a month number: 0
0 is not a valid month number

Enter a month number: 13
13 is not a valid month number

Enter a month number: two
'two' is not a valid month number

Enter a month number: 2
OK
```

Savings Account

Create a program called **savings_account** to compute compound interest for the initial deposit in a savings account.

Assume that

- The interest rate does not change after the initial deposit
- The capital income tax is subtracted from the interest before it is added to the balance.

First, the program inputs interest rate (%), capital income tax rate (%), the initial deposit, and the number of years. Then, the program should print the balance of the account at the end of each year.

Instruction

- You should ***compute the result in a simple for loop*** where you keep adding the yearly interest to the balance. If you try to use some pre-defined compound interest formula, it might give you slightly different results.
- If the last decimal on some line in your program's output differs from the example output, then please *change your calculation order*. Then retry.

Example output:

```
Enter interest rate: 5.0
Enter capital income tax rate: 30.0
Enter initial deposit: 1000
Enter number of years: 5

Year 1: 1035.00
Year 2: 1071.22
Year 3: 1108.72
Year 4: 1147.52
Year 5: 1187.69
```

In the example above, 35 is added to the balance at the end of the first year.

5 % of 1000 is 50
30 % of 50 is 15
50 - 15 = 35

Note: You are not required to validate any input in this exercise.

Double Factorial

The double factorial of ***n*** is the product of all the integers from 1 up to ***n*** that have the same parity (odd or even) as ***n***. You can use an arithmetic operator to determine the parity of an integer.

`0!! is 1` `5!! = 1 x 3 x 5 = 15` `6!! = 1 x 2 x 4 x 6 = 48`

Create a program called **double_factorial** that inputs a non-negative integer from the user. The program should display the *double factorial* of the inputted integer. If the input value is negative, the program should display "**Please enter a non-negative integer**".

Example output:

```
Enter a non-negative integer: 6
6!! = 48
```